



ALFIDO TECH

RIDING THE WAVES OF TECHNOLOGICAL
ADVANCEMENT

IRIS FLOWER CLASSIFICATION USING MACHINE LEARNING

Alfido Tech Machine Learning Internship

Submitted By

Name: Rahul Bisht

Candidate ID: BS/REG/122965

Internship: Machine Learning Internship

Organization: Alfido Tech

Project: Iris classification using Machine Learning

Submission Date: 20-07-2026

Certificate / Declaration

I hereby declare that the project entitled “Iris Flower Classification using Machine Learning” has been completed by me during my Machine Learning Internship at Alfido Tech. The work presented in this report is based on my own implementation and understanding of machine learning concepts, and all resources used have been properly acknowledged.

Rahul Bisht Machine Learning Intern, Alfido Tech

Acknowledgement

I sincerely thank Alfido Tech for providing me with the opportunity to complete this internship and gain practical experience in machine learning. I also acknowledge the support of open-source tools such as Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn, which were used throughout this project.

Table of Contents

- 1. Introduction**
- 2. Project Objectives**
- 3. Dataset Description**
- 4. Technology Stack**
- 5. Exploratory Data Analysis (EDA)**
- 6. Data Preprocessing**
- 7. Feature Engineering**
- 8. Machine Learning Models**
- 9. Model Evaluation**
- 10. Prediction Example**
- 11. Model Saving**
- 12. Applications**
- 13. Future Scope**
- 14. Conclusion**
- 15. References**
- 16. Appendix**

1. Introduction

Machine Learning enables computers to learn patterns from data and make predictions without explicit programming. Classification is a supervised learning technique used to assign observations to predefined categories.

The Iris Flower Classification dataset, introduced by Ronald A. Fisher, contains measurements of three flower species: Iris Setosa, Iris Versicolor, and Iris Virginica. Each sample includes four numerical features: Sepal Length, Sepal Width, Petal Length, and Petal Width.

The objective of this project is to build a machine learning model capable of accurately predicting the species of an Iris flower using these measurements.

2. Project Objectives

- Understand the Iris dataset.
- Perform Exploratory Data Analysis (EDA).
- Visualize relationships among features.
- Train multiple classification models.
- Compare model performance.
- Save the best-performing model for future predictions.

3. Dataset Description

The dataset contains 150 records, 4 numerical features, and 3 target classes. It is balanced, with 50 samples for each species and no missing values.

Property	Value
Dataset Name	Iris Classification Dataset
Total Records	150
Total Features	4
Target Variable Species	
Problem Type	Multi-Class Classification
Missing Values	None.

4. Technology Stack

Tool	Purpose
Python	Programming Language
Google Colab	Development Environment
Pandas	Data Manipulation
NumPy	Numerical Operations

Tool	Purpose
Matplotlib	Visualization
Seaborn	Statistical Visualization
Scikit-learn	Machine Learning Models
Joblib	Model Saving

5. Exploratory Data Analysis (EDA)

EDA was performed to understand the structure and distribution of the dataset. The following analyses were conducted:

- Dataset preview using head()**
- Dataset information using info()**
- Statistical summary using describe()**
- Missing value analysis**
- Histogram plots**
- Box plots**
- Pair plots**

(Dataset Head and Dataset Info)

```
[ ]  # Load the Iris dataset from the CSV file
df = pd.read_csv("iris.csv")

# Display the first five rows of the dataset
df.head()
```

▼ ...

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

▼ Step 3: Exploratory Data Analysis (EDA)

```
[ ] # Display dataset information
df.info()
```

▼

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```


(Describe Output and Missing Values)

```
[ ]  # Display the statistical summary of the dataset  
df.describe()
```

...

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

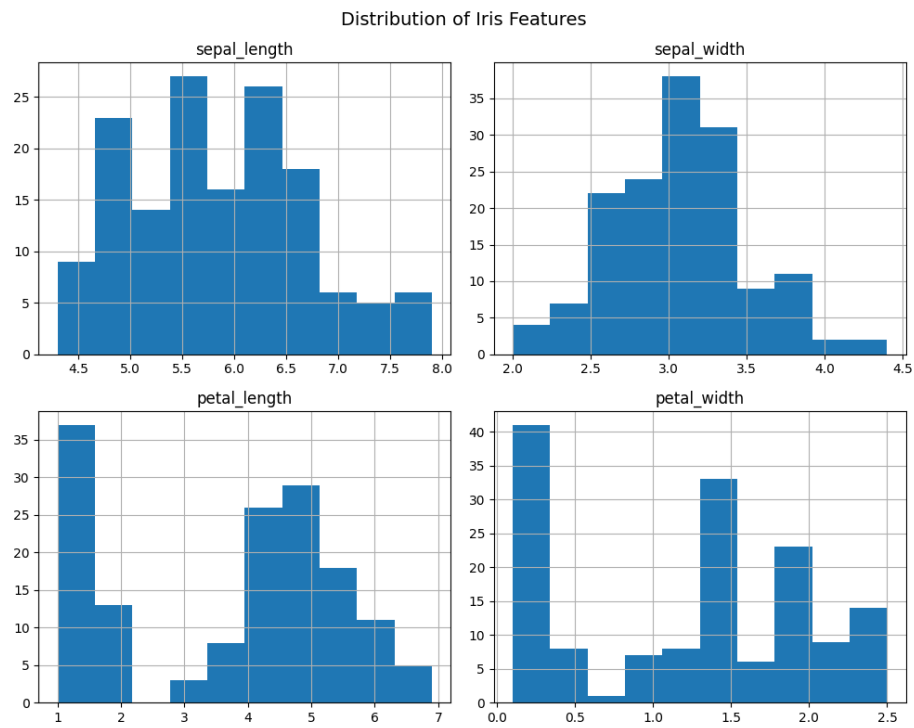
```
[ ] # Check for missing values in the dataset  
df.isnull().sum()
```

0

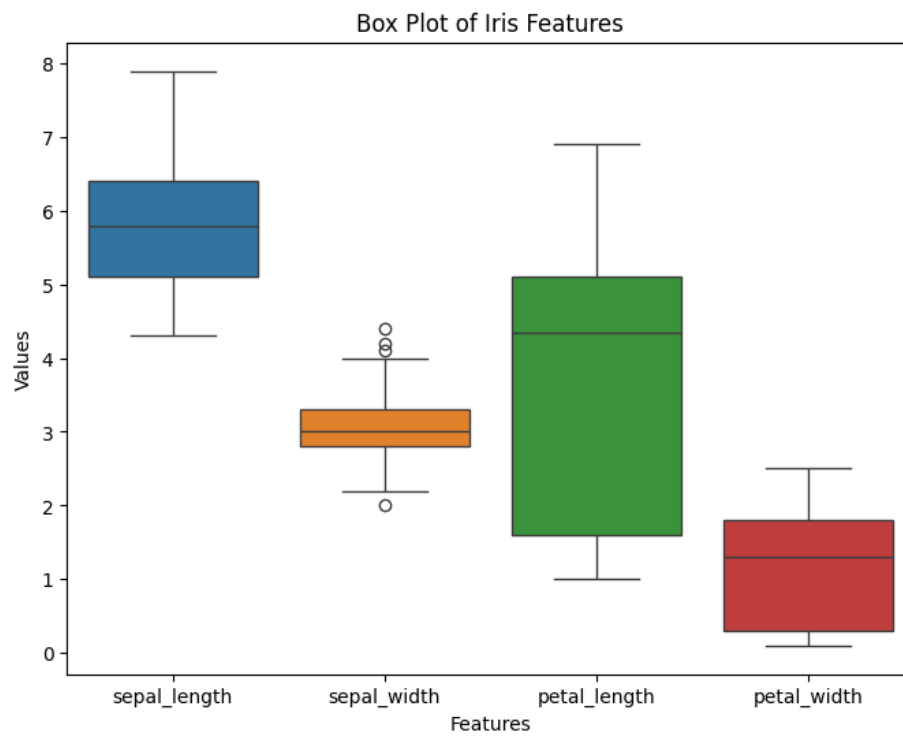
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

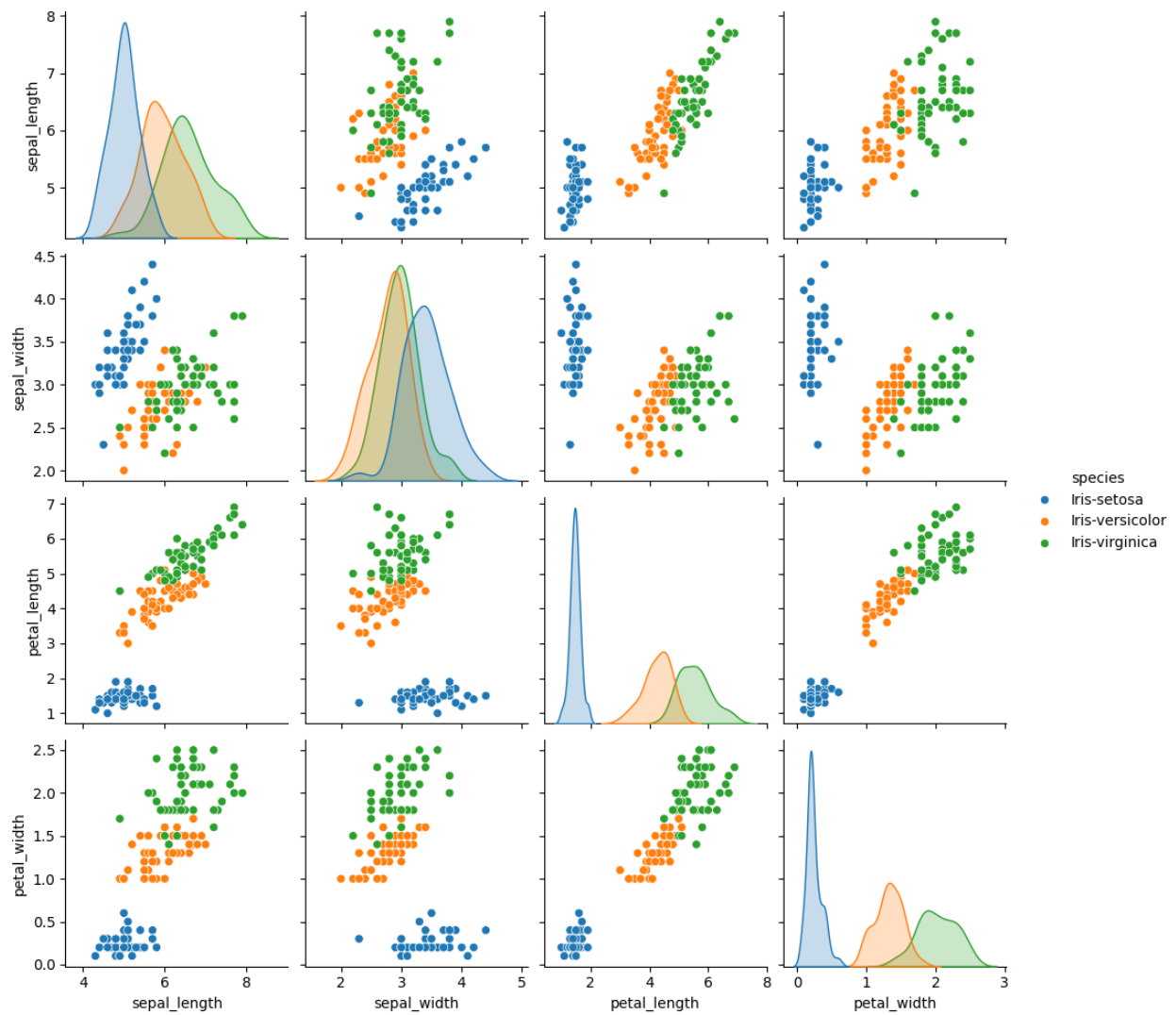
(Histogram)



(Box Plot)



(Pair Plot)



EDA Summary: The dataset is clean, balanced, and well suited for classification. Petal measurements provide strong separation among species.

6. Data Preprocessing

Since the Iris dataset is already clean, only minimal preprocessing was required.

- Missing values checked and confirmed absent.
- Numerical features selected as input variables.
- Species selected as the target variable.
- Dataset split into 80% training and 20% testing sets.

(Features and Target Variable)

```
... Input Features:
      sepal_length  sepal_width  petal_length  petal_width
0           5.1           3.5           1.4           0.2
1           4.9           3.0           1.4           0.2
2           4.7           3.2           1.3           0.2
3           4.6           3.1           1.5           0.2
4           5.0           3.6           1.4           0.2

Target Variable:
0  Iris-setosa
1  Iris-setosa
2  Iris-setosa
3  Iris-setosa
4  Iris-setosa
Name: species, dtype: object
```

7. Feature Engineering

The dataset already contains meaningful numerical features; therefore, only feature selection and train-test splitting were required before model training.

8. Machine Learning Models

8.1 Logistic Regression

A simple and fast baseline classifier that estimates class probabilities.

Advantages:

- Simple implementation
- Fast training
- Good interpretability

(Logistic Regression Output)

```
... Classification Report:
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

8.2 K-Nearest Neighbors (KNN)

KNN classifies samples based on the nearest training observations using distance measurements.

Advantages:

- Easy to understand
- Works well on small datasets
- Captures non-linear patterns

(KNN Output) kNN Model Accuracy: 100.00%

8.3 Decision Tree Classifier

Decision Tree classifies data by recursively splitting features into decision rules.

Advantages:

- Easy to visualize
- Handles non-linear relationships
- Requires minimal preprocessing

(Logistic Regression , k-NN Accuracy and Decision Tree Output)

Model Performance Comparison	

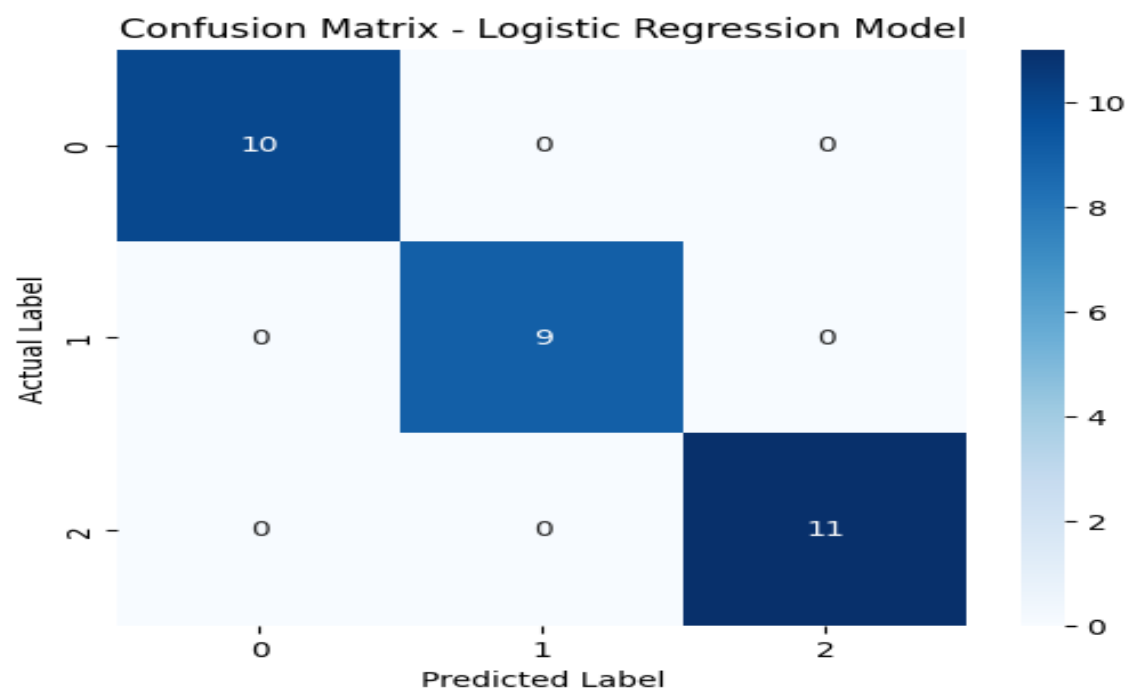
Logistic Regression Accuracy	: 100.00%
k-NN Accuracy	: 100.00%
Decision Tree Accuracy	: 100.00%

9. Model Evaluation

The models were evaluated using Accuracy, Precision, Recall, and F1-Score.

Model	Accuracy
Logistic Regression	100.00%
K-Nearest Neighbors	100.00%
Decision Tree	100.00%

(Confusion Matrix)



(Classification Report)

```
... Classification Report:
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	1.00	1.00	9
Iris-virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Best Model: The model with the highest accuracy and strongest overall classification performance was selected and saved for deployment.

10. Prediction Example and Prediction Output

```
# Load the saved Iris classification model

loaded_model = joblib.load("best_iris_model.pkl")

# Create a new flower sample for prediction

sample_flower = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=X.columns
)

# Predict the flower species using the loaded model

prediction = loaded_model.predict(sample_flower)

# Display the predicted Iris species

print("Predicted Iris Species:", prediction[0])

Predicted Iris Species: Iris-setosa
```

11. Model Saving

The trained model was saved using Joblib for future predictions.

```
joblib.dump(best_model, "best_iris_model.pkl")
```

12. Applications

- Botanical research
- Agricultural analysis
- Machine learning education
- Pattern recognition systems

13. Future Scope

- **Hyperparameter tuning**
- **Cross validation**
- **Web deployment using Flask or Streamlit**
- **Testing additional algorithms such as SVM and Random Forest**

14. Conclusion

This project successfully developed a machine learning model for classifying Iris flower species using supervised learning techniques. The workflow included data analysis, preprocessing, model training, evaluation, and model saving.

The results demonstrate that the Iris dataset is highly suitable for classification tasks and that the selected model can accurately predict flower species using sepal and petal measurements. The project also provided practical experience in building and evaluating machine learning classification systems.

15. References

- **Scikit-learn Documentation**
- **Pandas Documentation**
- **NumPy Documentation**
- **Matplotlib Documentation**
- **Seaborn Documentation**

- Google Colab
- Kaggle Iris Classification Dataset
- GitHub Repository

16. Appendix

GitHub Repository:

<https://github.com/rahul4bisht/Iris-Classification>

Google Colab Notebook:

(https://colab.research.google.com/drive/1hdBY1RDLRgDMC99OdLauuaxIkrt4rXM0?usp=drive_link)

Dataset Link:

<https://www.kaggle.com/datasets/bhanupratapbiswas/iris-classification-dataset>